

BIDA ML Starter

Stand 17.09.2026 Aufgaben Forecasting · Klassifikation · Regression

Umgebung lokal mit uv oder conda, Ergebnisse nach Snowflake

IN EINER STUNDE STARTKLAR

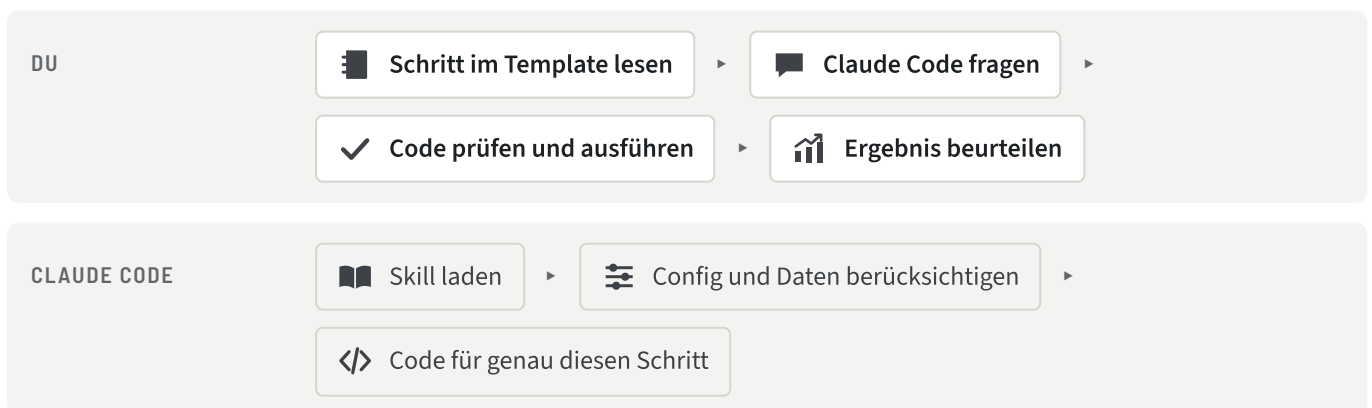
So kommst du zu deinem ersten Modell

- 1 Repository klonen**
`git clone <repo-url>`, dann den Ordner `BIDA-ML-Starter` in VS Code öffnen.
- 2 Umgebung erstellen**
Mit uv: `uv sync --extra cpu`, mit NVIDIA-GPU `--extra gpu`. Mit conda:
`conda env create -f environment.yml`.
- 3 Zugangsdaten anlegen**
`.env.example` nach `.env` kopieren, Account, User und eine Auth-Methode eintragen.
- 4 Smoke-Test ausführen**
Kernel `.venv` wählen und `00_environment_check.ipynb` ausführen. Alle Zellen müssen fehlerfrei enden.
- 5 Arbeits-Notebook anlegen**
Zum Beispiel `notebooks/work_<projekt>.ipynb`. Daneben das passende Template öffnen.
- 6 Claude Code nach dem Code fragen**
Pro Schritt eine Frage, zum Beispiel: Schritt 8 aus 02_forecasting für meine Daten.
- 7 Ergebnisse nach Snowflake schreiben**
`write_to_snowflake(df, "TABELLE")`
schreibt in die ML-Datenbank deiner Umgebung, heute `DEV_ML`.

DAS PRINZIP

Anleitung lesen, Code von Claude, Urteil von dir

Die Template-Notebooks enthalten bewusst keinen Code, sondern beschreiben, was zu tun ist und warum. Den Code schreibt Claude Code auf Basis der Skills. Du prüfst ihn und entscheidest.

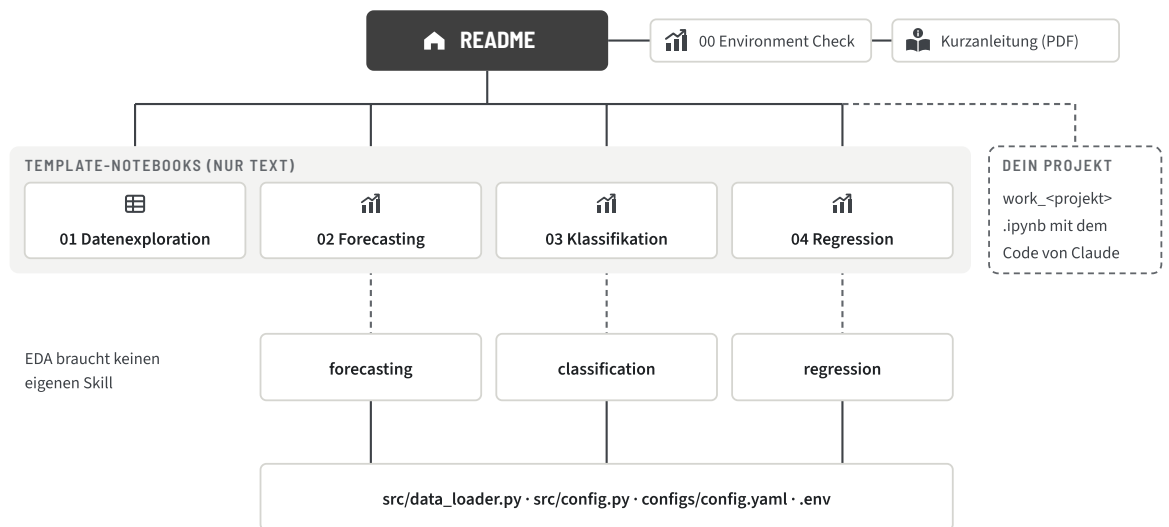


Der Code gehört in dein Arbeits-Notebook. Template-Notebooks und `src/` bleiben unverändert.

So ist das Projekt aufgebaut

EBENE 4

Infrastruktur



----- Notebook und Skill gehören als Paar zusammen

Die Skills liegen unter `.claude/skills/<name>/SKILL.md`. Claude Code erkennt sie automatisch. Mit `/forecasting`, `/classification` oder `/regression` lädst du einen Skill gezielt.

Die Bausteine

 Template-Notebooks <code>notebooks/</code> .Anleitung Schritt für Schritt, ohne Code.	 Skills <code>.claude/skills/</code> .Code- Referenz für Claude zu jedem Schritt.	 Konfiguration <code>configs/config.yaml</code> .Ziel- Schemas und Quelltabellen.
 Zugangsdaten <code>.env</code> im Projekt-Root, nie in Git.	 Infrastruktur <code>src/</code> . Config laden, aus Snowflake lesen, nach Snowflake schreiben.	 Leitplanken <code>.claude/</code> Rules und Hooks. Sperren Secrets, destruktives SQL und <code>git push</code> .
 Modelle und Reports <code>models/</code> und <code>reports/figures/</code> . Modelle liegen nicht in Git.	 Berechtigungen <code>sql/</code> . Rechte der Rolle <code>ML_DEVELOPER</code> , Referenz für Admins.	 Qualität <code>tests/</code> und pre-commit mit ruff und nbstripout.

Die Templates

00 Environment Check

Smoke-Test, mit Code

Prüft Python, Libraries, PyTorch mit GPU, die Snowflake-Verbindung und die Config. Nach jedem Setup einmal ausführen.

Erwartet: Rolle ML_DEVELOPER, Warehouse CONSUMER, Datenbank DEV_ML.

01 Datenexploration

EDA, 8 Schritte

Überblick, fehlende Werte, Verteilungen, Korrelationen, zeitliche Muster, Ausreisser. Am Ende die Empfehlung, welches Template folgt.

Skill: keiner nötig, Claude schreibt den Code direkt aus der Anleitung.

02 Forecasting

Zeitreihen, 20 Schritte

Von einfach zu komplex: Baseline, statistische Modelle (AutoARIMA, ETS, Theta, SARIMAX), klassisches ML (LightGBM, XGBoost über MLForecast), neurale Modelle (N-HITS, TFT, TSMixerx, PatchTST). Dazu Cross-Validation, Fehleranalyse nach Wochentag, SHAP, Tuning mit Optuna.

Skill: forecasting. **Datenformat:** unique_id, ds, y. **Metriken:** MAE, RMSE, MAPE, sMAPE.

03 Klassifikation

11 Schritte

sklearn-Pipeline mit Preprocessing, LightGBM oder XGBoost, stratifizierte Cross-Validation, GridSearch, Evaluation mit Confusion Matrix, ROC- und PR-Kurve, Feature Importance mit SHAP.

Skill: classification. **Metriken:** Accuracy, F1, ROC-AUC, PR-AUC.

04 Regression

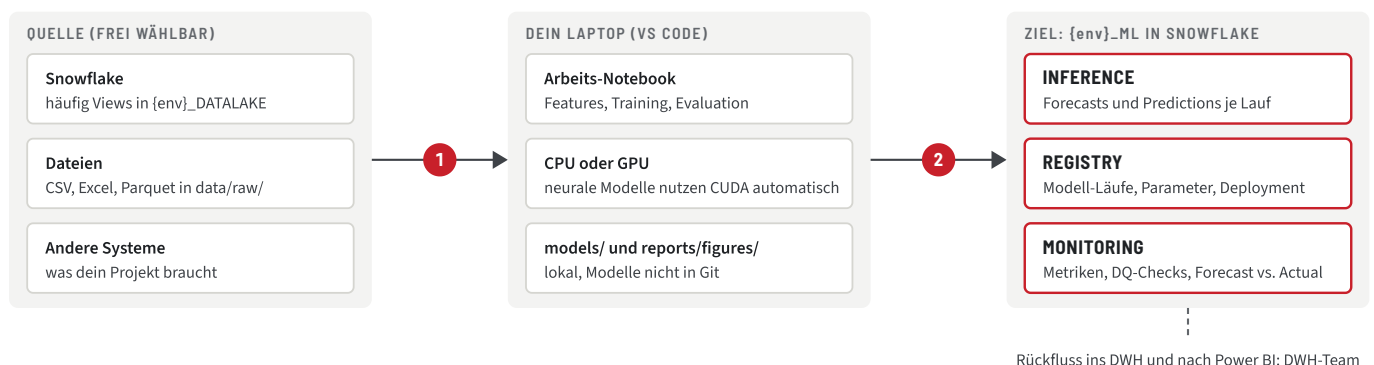
16 Schritte

Outlier-Analyse der Zielvariable, Pipeline, Baseline, Cross-Validation, Tuning mit Optuna, Actual gegen Predicted, Residuen-Analyse, SHAP.

Skill: regression. **Metriken:** RMSE, MAE, R^2 , MAPE.

DATEN

Woher die Daten kommen, wohin die Ergebnisse gehen

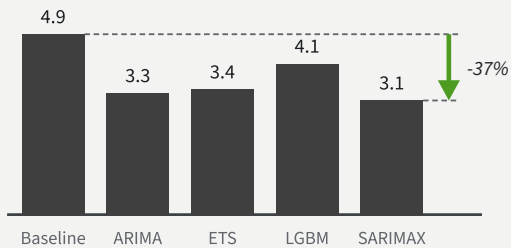


1 Laden. Aus Snowflake mit `load_query`, `load_table` oder `load_timeseries`. Dateien mit pandas, zum Beispiel `pd.read_parquet`.

2 Schreiben. Immer mit `write_to_snowflake`, immer anhängen statt überschreiben, immer mit `run_id`, Modellname, Version und Zeitstempel.

{env} kommt aus ML_ENV in deiner .env : DEV, INT oder PROD. Auf Laptops immer DEV.

Modelle richtig vergleichen



Beispiel: MAE je Modell auf dem Test-Set. Tiefer ist besser. Der Pfeil vergleicht das beste Modell mit der Baseline.

Die Baseline ist Pflicht

Zuerst ein einfaches Referenzmodell, zum Beispiel Seasonal Naive oder DummyRegressor. Jedes weitere Modell muss sie schlagen.

Der Split entscheidet

Zeitreihen immer **zeitlich** teilen, nie zufällig. Klassifikation immer **stratifiziert**. Sonst sind die Metriken zu optimistisch.

Nur bekannte Features

Exogene Features müssen in der Zukunft bekannt sein: Kalender, Feiertage, geplante Aktionen. Lag-Features nie aus der Zukunft.

Eine Zahl reicht nicht

Cross-Validation zeigt die Streuung, Fehleranalyse und Residuen zeigen, *wo* das Modell danebenliegt.

Vorsicht bei MAPE mit Nullwerten. Enthält die Zielgrösse Nullen oder sehr kleine Werte, explodiert der MAPE oder ist nicht definiert. Dann sMAPE oder MAE verwenden. Das ist kein Modellfehler, sondern eine Eigenschaft der Kennzahl.

CLAUDE CODE

So fragst du richtig

Nenne das Template, die Schritt-Nummer und was an deinen Daten besonders ist.

Gut: Schritt 7 aus 02_forecasting. Meine Daten sind täglich von Montag bis Freitag, Horizont 10 Tage, drei Serien. Exogene Features: Feiertage Kanton Luzern.

Gut: Die Baseline hat MAE 4.9, LightGBM 4.1. Analysiere die Fehler nach Wochentag (Schritt 14) und schlage Features vor.

Zu vage: Mach mir ein Forecast-Modell.



Skill gezielt laden

`/forecasting`,
`/classification`,
`/regression`. Sonst wählt Claude den Skill selbst.



Zuerst klärt Claude

Datenquelle, Zielvariable und Horizont. Fehlt etwas in der Config, weist Claude darauf hin.



Was Claude nicht darf

`.env` und Schlüssel lesen, DROP, DELETE oder TRUNCATE ausführen, `git push`. Das erzwingt ein Hook.

Gut zu wissen

Tools immer mit `python -m` starten. Auf den Pistor-Notebooks sind die Launcher-EXEs in `.venv/Scripts` blockiert, zum Beispiel `pytest.exe` und `pre-commit.exe`. Deshalb `uv run python -m pytest` und `uv run python -m pre_commit`. `uv run ruff` funktioniert direkt.

Firmen-Proxy

Der Proxy ersetzt TLS-Zertifikate. `pyproject.toml` setzt deshalb `system-certs = true`, damit uv die Zertifikate aus dem Windows-Speicher nutzt. Du musst nichts einstellen.

Schlüssel und `.env`

Die `.env` enthält nur den Pfad. Der private Schlüssel liegt unter `~/snowflake/rsa_key.p8` und verlässt den Laptop nie. Den öffentlichen Schlüssel hinterlegt der Snowflake-Admin. Bis dahin hilft `SF_AUTHENTICATOR=externalbrowser`.

Abhängigkeiten ändern

Nur in `pyproject.toml`. Danach `uv lock` und `uv export --no-hashes --group dev -o requirements.txt`, damit conda dieselben Versionen erhält. Beides committen.

Reproduzierbarkeit

`RANDOM_STATE = 42` aus `src.config` in jedem Modell und jeder Aufteilung. Trainierte Modelle unter `models/<name>_v<n>_<datum>/`.

GPU

`uv sync --extra gpu` installiert PyTorch für CUDA 13.0 und braucht einen NVIDIA-Treiber ab Version 580. Der Smoke-Test zeigt `CUDA: True`. LightGBM und XGBoost laufen auf der CPU.

Eine Umgebung, ein Ziel

`ML_ENV` steuert Ziel-Datenbank und Snowflake-Quellen gemeinsam. Nie `DEV_ML` oder `PROD_ML` fest in den Code schreiben. Geschrieben wird nur in `{env}_ML`, den Rückfluss ins DWH macht das DWH-Team.

Notebooks in Git

`nbstripout` entfernt beim Commit alle Outputs. So bleiben Diffs lesbar und es landen weder Daten noch Secrets im Repository.

Vom Experiment zum Projekt

Dieses Repository ist die Methoden-Bibliothek. Ein echtes Vorhaben mit Spec und Verifier startest du aus dem Agentic Engineering Starter Template. Es kopiert Skills, Notebooks, `src/` und `configs/` von hier.

ZUM NACHSCHLAGEN

Die wichtigsten Befehle

<code>uv sync --extra cpu</code>	Umgebung erstellen oder aktualisieren. Mit GPU: <code>--extra gpu</code> .
<code>conda env create -f environment.yml</code>	Dieselbe Umgebung mit conda, danach <code>conda activate bida-ml</code> .
<code>uv run python -m pytest</code>	Tests ausführen.
<code>uv run ruff check --fix src tests</code>	Python-Code prüfen und automatisch korrigieren.
<code>uv run python -m pre_commit install</code>	Einmalig: Git-Hooks für ruff und nbstripout aktivieren.
<code>git push</code>	Machst immer du selbst. Claude Code darf committen, aber nicht pushen.